

Vision-Based Traffic Analytics: A Modular Pipeline for Vehicle Detection, Tracking, and Intersection-Level Traffic Monitoring from Monocular Camera Footage

Manjul Balayar, Md Solaiman Hosen

Department of Computer Science, Iowa State University

Abstract

Efficient traffic monitoring at urban intersections is essential for informed signal timing, congestion management, and long-term transportation planning. Traditionally, municipalities have depended on hardware-based sensing infrastructure—including inductive loop detectors, radar units, and pneumatic tube counters—to collect vehicle counts, speeds, and lane occupancy data. While broadly deployed, these systems carry substantial drawbacks: high installation costs, ongoing maintenance burdens, limited spatial coverage, and an inability to capture the rich spatiotemporal information needed for fine-grained analytics. At the same time, the vast majority of signalized intersections are already equipped with surveillance cameras, presenting a compelling opportunity to extract structured traffic analytics directly from existing video feeds using modern computer vision techniques.

In this work, we propose a modular, vision-based traffic analytics pipeline that transforms raw monocular intersection footage into actionable traffic metrics. Our system is organized into three layers—perception, tracking, and analytics. In the perception layer, we employ YOLOv8, a state-of-the-art single-stage object detection model, to localize vehicles in each video frame with confidence thresholding and Non-Maximum Suppression as post-processing. The tracking layer formulates multi-object tracking as a recursive state estimation and data association problem: each vehicle is modeled by a state vector encoding position and velocity, estimated via a Kalman filter under a constant-velocity motion model, and associated with detections through an Intersection-over-Union cost matrix solved globally using the Hungarian algorithm. A track lifecycle management protocol handles initialization, update, and termination of vehicle identities. The analytics layer transforms stable trajectories into three complementary traffic metrics: flow rate estimation via virtual counting lines, lane utilization analysis through region-of-interest assignment, and congestion detection by monitoring average vehicle speeds against spatial density. We evaluate on the UA-DETRAC benchmark across both tracking-level metrics (MOTA, identity switches, track fragmentation) and analytics-level metrics (counting accuracy, lane assignment correctness, congestion detection consistency), and compare against degraded baselines to isolate each component’s contribution. Future directions include depth-aware speed estimation, multi-camera fusion, and real-time edge deployment optimization.

1 Introduction and Motivation

Traffic monitoring lies at the heart of modern urban transportation infrastructure. Reliable, real-time knowledge of vehicle volumes, speeds, directional flows, and congestion patterns is indispens-

able for a wide range of transportation engineering applications—from adaptive signal control and incident detection to long-range capacity planning and environmental impact assessment. As urban populations continue to grow and road networks become increasingly strained, the demand for accurate, scalable, and cost-effective traffic monitoring has never been greater.

1.1 Limitations of Traditional Sensing Infrastructure

For decades, traffic data collection has relied on dedicated hardware sensors—inductive loop detectors embedded in the pavement, radar units, microwave detectors, and pneumatic tube counters. However, these systems share fundamental limitations: high installation costs (loop detectors require cutting into the road surface, involving lane closures and specialized labor), ongoing maintenance burdens (loops are susceptible to pavement degradation and sensor failure), narrow spatial coverage (each sensor monitors only a single lane at a single point), and limited data richness (only aggregate scalar measurements such as volume counts and point speeds, lacking the spatiotemporal trajectory information needed for sophisticated analytics).

1.2 The Opportunity of Existing Camera Infrastructure

In contrast to the high cost and limited coverage of hardware sensors, a parallel infrastructure already exists at most urban intersections: surveillance cameras (Figure 1). Installed primarily for security monitoring and incident documentation, these cameras are present at the vast majority of signalized intersections worldwide.



Figure 1: Surveillance cameras mounted at a typical urban intersection, providing video feeds that can be repurposed for traffic analytics.

The video streams from these cameras contain far richer information than any point sensor—every frame encodes positions, sizes, and appearances of all visible vehicles, while the temporal sequence implicitly encodes motion and interaction patterns. This motivates the fundamental question driving our work: *can we transform raw monocular surveillance camera footage into structured, quantitative traffic analytics using computer vision, thereby repurposing existing infrastructure and eliminating the need for additional hardware sensors?*

Recent advances in deep learning–based object detection (e.g., the YOLO family) and multi-object tracking suggest that a well-designed vision-based pipeline can approach or exceed the data quality provided by traditional sensors, while offering dramatically lower deployment costs and richer spatiotemporal information.

1.3 Target Audience and User Population

The proposed system targets several key stakeholder groups. The primary users are **municipal traffic engineers and transportation departments**, who require signal timing optimization, intersection safety analysis, and traffic management data but currently rely on costly sensor installations or periodic manual counts. A secondary audience includes **urban planners and smart city initiatives** requiring long-term traffic pattern data for infrastructure investment and sustainability planning. Additionally, **transportation researchers** studying traffic flow theory and driver behavior would benefit from the rich trajectory-level data our system produces. By lowering the cost of obtaining structured traffic data, our system makes analytics accessible to smaller municipalities that cannot justify traditional sensor infrastructure.

1.4 Technical Challenges

Building a reliable vision-based traffic analytics system involves several non-trivial challenges. **Robust Vehicle Detection** is complicated by partial occlusion, varying lighting conditions (shadows, glare, nighttime), and diverse vehicle appearances. **Consistent Multi-Object Tracking** requires maintaining vehicle identities across time despite dense traffic, missed detections, and visually similar vehicles. **Trajectory-to-Metric Transformation** demands carefully designed spatial logic including virtual counting lines and lane-specific regions of interest. Finally, **Real-Time Processing Constraints** impose computational limits on algorithm complexity for practical deployment.

1.5 Project Goals and Scope

The goal of this project is to design, implement, and evaluate a modular, end-to-end vision-based traffic analytics pipeline. The system takes raw monocular traffic surveillance video as input and produces structured traffic metrics including vehicle flow rates, lane utilization distributions, and congestion detection events. The pipeline is organized into three decoupled layers:

1. **Perception Layer:** A deep learning–based object detector (YOLOv8) localizes vehicles in each frame, producing bounding boxes with confidence scores.
2. **Tracking Layer:** A Kalman filter–based multi-object tracker with Hungarian algorithm data association maintains consistent vehicle identities across frames.
3. **Analytics Layer:** Trajectory-based modules compute flow rates, lane utilization, and congestion indicators using configurable spatial primitives.

2 Related Work

2.1 Vision-Based Traffic Monitoring

The use of computer vision for traffic surveillance has been explored for over two decades [1, 2]. Early systems relied on background subtraction methods—such as Gaussian Mixture Models (GMMs) [3]—to segment moving vehicles from a static camera background, followed by connected-component analysis or blob tracking. While effective under controlled conditions, these classical approaches are sensitive to illumination changes and slow-moving or stopped vehicles.

The emergence of deep learning-based object detectors has substantially improved robustness. The YOLO (You Only Look Once) family [4], progressing through YOLOv3 [5], YOLOv5 [6], and YOLOv8 [7], has been widely adopted in traffic applications due to its favorable speed–accuracy tradeoff. These single-stage detectors process the entire image in a single forward pass, enabling real-time inference. Alternative two-stage detectors such as Faster R-CNN [8] offer higher accuracy on certain benchmarks but at greater computational cost, making them less suitable for real-time video processing.

Azimjonov et al. [9] proposed a real-time traffic flow monitoring system for road intersections combining deep object detection with counting heuristics, showing that vision-based flow estimation can approach the accuracy of inductive loop detectors. However, many systems rely on simple frame-by-frame counting rather than principled tracking, limiting accuracy under occlusion.

2.2 Multi-Object Tracking

Multi-object tracking (MOT) aims to maintain consistent identities for detected objects across video frames [10]. The SORT (Simple Online and Realtime Tracking) algorithm [11] combines a Kalman filter [12] for linear motion prediction with the Hungarian algorithm [13] for global assignment based on IoU overlap. SORT achieves competitive performance at extremely high speed, making it one of the most widely used baselines.

DeepSORT [14] extends SORT by incorporating a deep appearance descriptor into the association cost, enabling re-identification after prolonged occlusions. More recent approaches such as ByteTrack [15] improve association recall by considering low-confidence detections in a second association pass. For this project, we adopt a SORT-style framework motivated by its simplicity, real-time capability, and strong performance on traffic surveillance benchmarks where motion patterns are relatively predictable.

2.3 Traffic Analytics from Trajectories

Prior work has demonstrated trajectory-based approaches for vehicle counting via virtual tripwires, speed estimation from frame-to-frame displacement, and turning-movement analysis [16, 17]. Lane utilization has been derived by assigning trajectories to predefined lane regions, and congestion detection has been approached through average trajectory speed and spatial density measures. A key insight is that the reliability of all trajectory-based analytics depends critically on tracking quality: identity switches cause double-counting, and track fragmentation splits a single vehicle into multiple partial counts.

3 Team Experience

3.1 Md Solaiman Hosen

Md Solaiman Hosen has completed coursework directly relevant to this project: **COMS 5730: Machine Learning**, **COMS 5740: Introduction to Machine Learning**, and **COMS 5750: Computational Perception**. Through these courses, he has developed a strong understanding of deep learning architectures (including CNNs for image classification and object detection), image processing and filtering techniques, and feature extraction pipelines. His programming proficiency spans Python, Java, C++, and Kotlin, with hands-on experience in PyTorch, OpenCV, scikit-learn, and Matplotlib.

3.2 Manjul Balayar

Manjul Balayar has completed several courses directly relevant to this project, including **COMS 5730: Machine Learning**, **COMS 5740: Introduction to Machine Learning**, **COMS 5750: Computational Perception**, **COMS 5790: Natural Language Processing**, and **EE 526: Deep Learning**. Through these courses, he has developed a strong foundation in machine learning and deep learning methods, including convolutional neural networks for computer vision tasks such as image classification and object detection. His coursework in EE 526 provided deeper exposure to modern deep learning architectures and training methodologies used in vision applications. He has experience implementing machine learning pipelines using **Python** and common ML libraries such as **PyTorch**, **OpenCV**, **scikit-learn**

3.3 Why Our Team Can Succeed

Our coursework in machine learning and computational perception, combined with hands-on experience in deep learning frameworks and computer vision libraries, provides a strong foundation for this project. The algorithms we propose—YOLOv8 detection, Kalman filtering, and Hungarian assignment—are well-documented and supported by mature open-source implementations, reducing implementation risk. Our primary contribution is integrating these components into a coherent end-to-end pipeline with a principled evaluation framework.

4 Problem Definition

4.1 Input Specification

The system takes as input a monocular traffic surveillance video sequence $\mathcal{V} = \{I_1, I_2, \dots, I_T\}$, where each $I_t \in \mathbb{R}^{H \times W \times 3}$ is an RGB frame captured at time step t . The camera is assumed to be stationary with a roughly overhead or oblique viewing angle typical of intersection-mounted cameras. No intrinsic or extrinsic camera calibration parameters are assumed.

4.2 Output Specification

The system produces the following structured outputs:

- **Vehicle Flow Rate:** The number of vehicles passing through user-defined counting lines per unit time, reported in vehicles per minute (veh/min), with optional disaggregation by direction or lane.
- **Lane Utilization Statistics:** The distribution of traffic load across individual lanes, computed as the fraction of total tracked vehicles assigned to each lane-specific region of interest.
- **Congestion Indicators:** Binary or graded congestion flags derived from the co-occurrence of low average vehicle speed and high spatial density of active tracks.

4.3 Sub-Tasks

Producing the above outputs from raw video requires three sequential sub-tasks:

1. **Vehicle Detection:** For each frame I_t , produce a set of detections $\mathcal{D}_t = \{(b_i^t, s_i^t, c_i^t)\}$, where $b_i^t \in \mathbb{R}^4$ is a bounding box, $s_i^t \in [0, 1]$ is a confidence score, and c_i^t is a class label.
2. **Multi-Object Tracking:** Associate detections across frames to produce tracks $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_K\}$, where each track records estimated bounding box and velocity at each active frame.
3. **Traffic Analytics:** Transform the set of tracks $\mathcal{T}$ into the structured traffic metrics defined above.

5 Proposed Approach

5.1 System Architecture

The proposed system implements a three-layer pipeline that processes traffic surveillance video through perception, tracking, and analytics stages. Figure 2 illustrates the overall architecture and dataflow of the system.

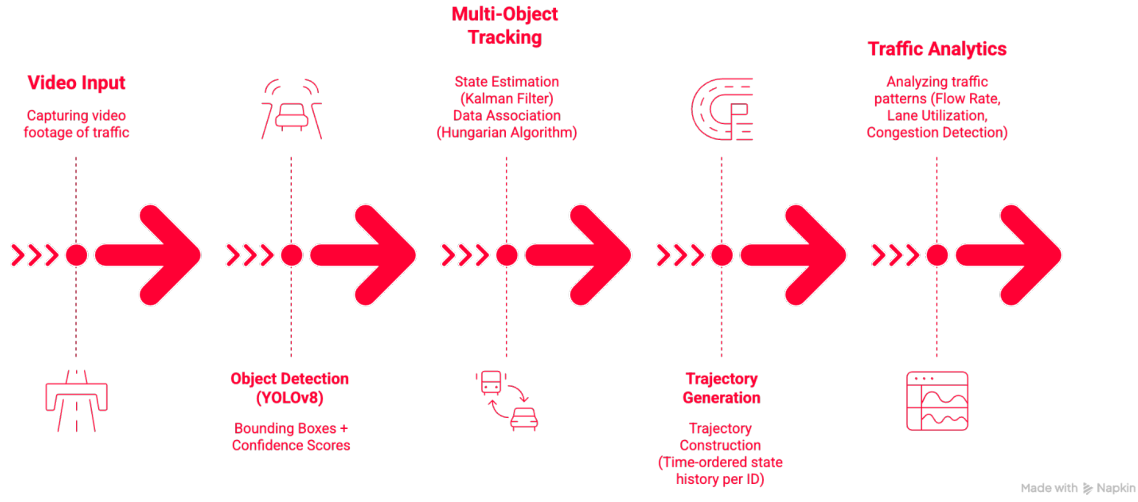


Figure 2: Overall pipeline architecture of the proposed traffic analytics system. Video input is processed sequentially through object detection (YOLOv8), multi-object tracking (Kalman filter with Hungarian algorithm), trajectory generation, and traffic analytics modules.

The architecture is modular: each layer communicates through well-defined interfaces (detections, tracks, metrics), allowing independent replacement or upgrade. Each video frame passes to YOLOv8 for detection, then to the Kalman-based tracker for identity association, and finally to the analytics module for metric computation.

5.2 Equipment and Software

Table 1 summarizes the hardware and software resources required for this project.

Table 1: Equipment and software used in the project.

Category	Details
GPU	NVIDIA GPU with CUDA support (e.g., RTX 3060 or higher)
CPU	Multi-core processor (Intel i7 / AMD Ryzen 7 or equivalent)
RAM	16 GB minimum
Storage	50+ GB for dataset and model weights
Language	Python 3.10+
Detection	Ultralytics YOLOv8 (pre-trained COCO weights)
Deep Learning	PyTorch 2.x with CUDA support
Computer Vision	OpenCV 4.x (video I/O, drawing, geometric operations)
Tracking	NumPy, SciPy (Kalman filter, Hungarian algorithm)
Visualization	Matplotlib, OpenCV (trajectory and analytics overlays)
Data Handling	Pandas (metrics logging and analysis)
Development	Git (version control), Jupyter Notebooks (prototyping)

5.3 Object Detection with YOLOv8

The perception layer employs YOLOv8 [7], a single-stage object detector that processes the entire input image in a single forward pass, directly regressing bounding box coordinates and class probabilities. We use a pre-trained YOLOv8 model and focus on vehicle-relevant classes (car, truck, bus, motorcycle). Two post-processing steps refine predictions:

- **Confidence Thresholding:** Detections with confidence scores below a threshold θ_{conf} are discarded to suppress weak false positives.
- **Non-Maximum Suppression (NMS):** Among overlapping detections, only the highest-confidence box is retained. Boxes with IoU overlap exceeding θ_{NMS} relative to a higher-scoring box are suppressed.

Detection quality directly governs downstream tracking: false positives create spurious tracks, missed detections cause fragmentation, and noisy bounding boxes degrade Kalman filter motion estimation.

5.4 Multi-Object Tracking

The tracking layer is the core algorithmic component of the pipeline. We formulate multi-object tracking as a joint state estimation and data association problem, following the SORT paradigm [11].

5.4.1 State Representation and Motion Model

Each tracked vehicle k is represented by a state vector:

$$\mathbf{x}_k = [x, y, w, h, \dot{x}, \dot{y}]^\top \quad (1)$$

where (x, y) is the bounding box center, (w, h) are the width and height, and $(\dot{x}, \dot{y})$ are the velocity components. The constant-velocity state transition is:

$$\mathbf{x}_k^{t|t-1} = \mathbf{F} \mathbf{x}_k^{t-1|t-1} + \mathbf{w}_k \quad (2)$$

where $\mathbf{F}$ is the state transition matrix and $\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q})$ is process noise.

5.4.2 Kalman Filter Prediction and Update

At each time step, the Kalman filter [12] executes a predict–correct cycle for every active track, as illustrated in Figure 3.

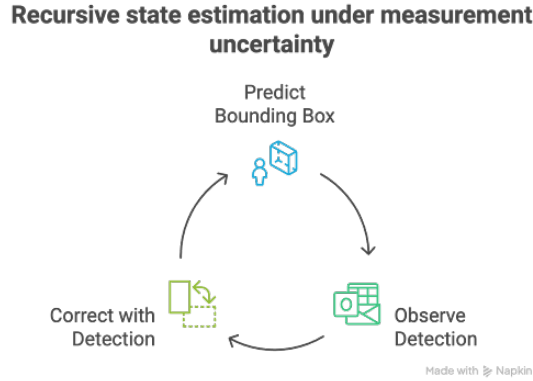


Figure 3: The recursive predict–observe–correct cycle of the Kalman filter used for vehicle state estimation under measurement uncertainty.

Prediction: The filter propagates the state estimate and uncertainty forward:

$$\hat{\mathbf{x}}_k^{t|t-1} = \mathbf{F} \hat{\mathbf{x}}_k^{t-1|t-1} \quad (3)$$

$$\mathbf{P}_k^{t|t-1} = \mathbf{F} \mathbf{P}_k^{t-1|t-1} \mathbf{F}^\top + \mathbf{Q} \quad (4)$$

Update: When a detection $\mathbf{z}_k^t$ is associated with track k , the filter corrects the prediction:

$$\mathbf{K}_k^t = \mathbf{P}_k^{t|t-1} \mathbf{H}^\top \left(\mathbf{H} \mathbf{P}_k^{t|t-1} \mathbf{H}^\top + \mathbf{R} \right)^{-1} \quad (5)$$

$$\hat{\mathbf{x}}_k^{t|t} = \hat{\mathbf{x}}_k^{t|t-1} + \mathbf{K}_k^t \left(\mathbf{z}_k^t - \mathbf{H} \hat{\mathbf{x}}_k^{t|t-1} \right) \quad (6)$$

$$\mathbf{P}_k^{t|t} = (\mathbf{I} - \mathbf{K}_k^t \mathbf{H}) \mathbf{P}_k^{t|t-1} \quad (7)$$

where $\mathbf{H}$ is the observation matrix, $\mathbf{R}$ is measurement noise covariance, and $\mathbf{K}_k^t$ is the Kalman gain.

5.4.3 Data Association via the Hungarian Algorithm

When multiple vehicles are present, we solve the data association problem as a global assignment using the Hungarian algorithm [13]:

1. **Cost Matrix Construction:** For each predicted track k and detection j , compute $C_{kj} = 1 - \text{IoU}(\hat{b}_k^{t|t-1}, b_j^t)$.
2. **Optimal Assignment:** Apply the Hungarian algorithm to find the assignment minimizing total cost. Assignments with cost exceeding θ_{match} are rejected.

5.4.4 Track Lifecycle Management

- **Initialization:** Unmatched detections initialize tentative tracks, promoted to active after n_{init} consecutive matches.
- **Update:** Matched tracks are corrected via the Kalman filter update step.
- **Termination:** Tracks unmatched for n_{lost} consecutive frames are terminated.

5.5 Trajectory Generation

For each active track τ_k , the system stores the time-indexed sequence of Kalman-corrected state estimates:

$$\tau_k = \left\{ \left(t, \hat{\mathbf{x}}_k^{t|t} \right) \mid t \in [t_k^{\text{start}}, t_k^{\text{end}}] \right\} \quad (8)$$

These trajectories encode the complete spatiotemporal path of each vehicle and serve as input to the analytics layer.

5.6 Traffic Analytics

The analytics layer computes three categories of traffic metrics from vehicle trajectories, as visualized in Figure 4.

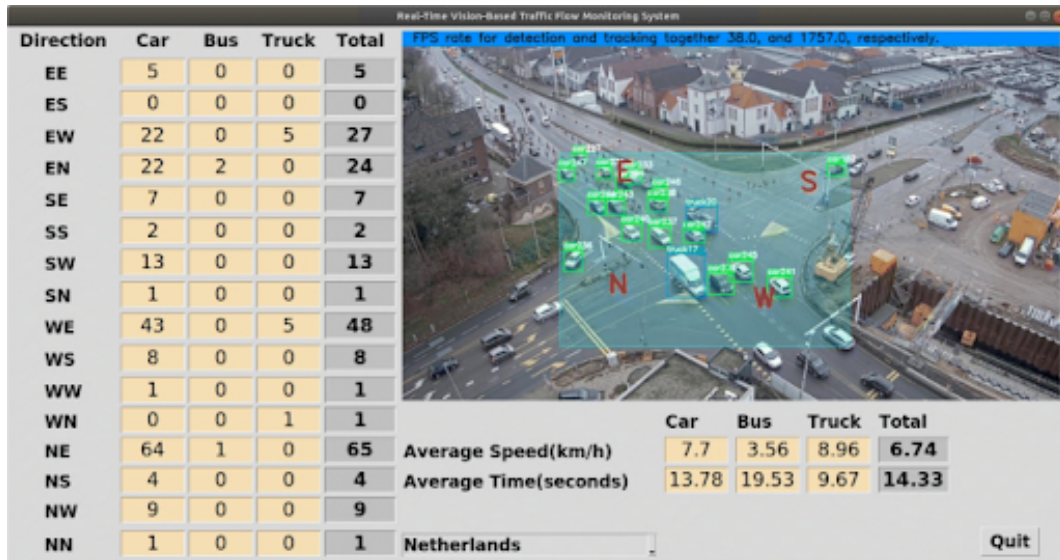


Figure 4: Example output of a real-time vision-based traffic flow monitoring system showing directional vehicle counts, vehicle type classification, average speed, and intersection-level analytics overlaid on a live camera feed.

5.6.1 Flow Rate Estimation

Virtual counting lines are defined at user-specified positions across the intersection. For each trajectory, the system tests whether consecutive position estimates cross a counting line. Flow rate is computed as:

$$\text{Flow}(l, \Delta t) = \frac{N_{\text{crossings}}(l, \Delta t)}{\Delta t} \quad (9)$$

where $N_{\text{crossings}}(l, \Delta t)$ is the number of trajectories crossing line l within window Δt .

5.6.2 Lane Utilization

Lane-specific regions of interest (ROIs) are defined as polygonal areas. Each trajectory is assigned to the lane whose ROI contains the majority of its position estimates:

$$U(l, \Delta t) = \frac{N_{\text{vehicles}}(l, \Delta t)}{\sum_{l'} N_{\text{vehicles}}(l', \Delta t)} \quad (10)$$

5.6.3 Congestion Detection

Average speed within a region is computed from Kalman state velocity components:

$$\bar{v}(R, t) = \frac{1}{|\mathcal{T}_R^t|} \sum_{k \in \mathcal{T}_R^t} \sqrt{\dot{x}_k^2 + \dot{y}_k^2} \quad (11)$$

A congestion event is flagged when average speed drops below $v_{\min}$ and density exceeds $\rho_{\max}$:

$$\text{Congestion}(R, t) = \mathbb{I}[\bar{v}(R, t) < v_{\min} \wedge \rho(R, t) > \rho_{\max}] \quad (12)$$

Temporal smoothing via a sliding window prevents transient slowdowns from triggering false alerts.

6 Dataset

We evaluate our pipeline on the UA-DETRAC (University at Albany Detection and Tracking) benchmark [18]. The dataset consists of over 100 video sequences recorded at real-world traffic intersections, comprising approximately 140,000 annotated frames with over 1.21 million labeled bounding boxes. Videos are captured at 25 fps with 960×540 resolution.

A key strength of UA-DETRAC is its diversity across multiple dimensions relevant to real-world deployment, as illustrated in Figure 5:

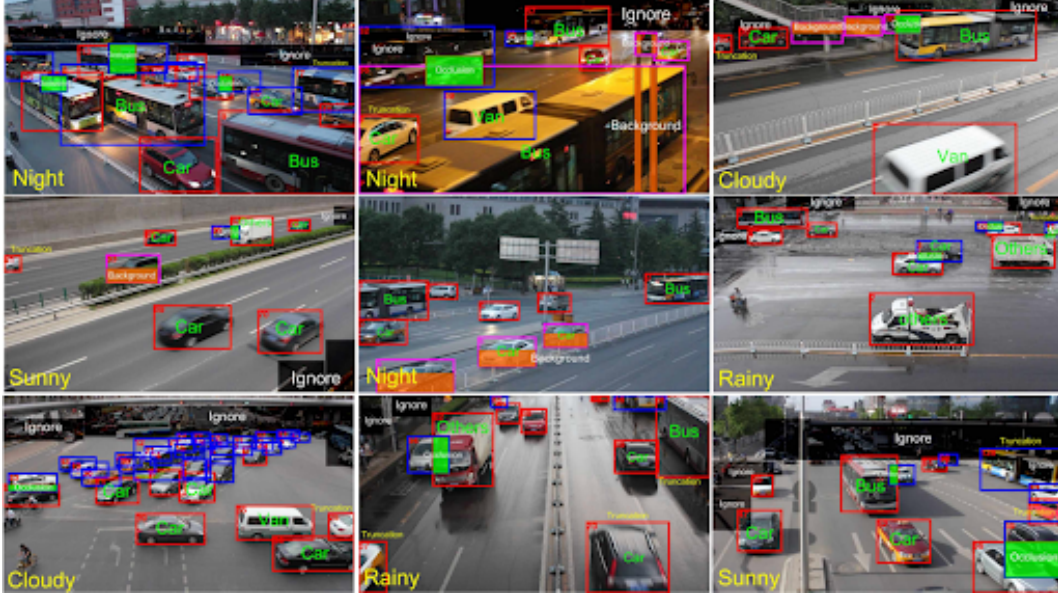


Figure 5: Example annotated frames from the UA-DETRAC dataset showing vehicle bounding boxes under diverse conditions: sunny, cloudy, rainy, and nighttime scenes with varying traffic densities and vehicle types.

- **Weather Conditions:** Sunny, cloudy, rainy, and nighttime sequences enable evaluation of robustness to illumination variation and reduced visibility.
- **Traffic Density:** Sequences range from sparse free-flow to dense congested conditions, testing tracker identity maintenance under varying occlusion levels.
- **Vehicle Types:** The dataset contains cars, buses, trucks, and vans, providing diversity in vehicle scale and appearance.
- **Camera Viewpoints:** Multiple intersection configurations and camera mounting angles test generalization across geometric setups.

Each frame is annotated with axis-aligned bounding boxes along with attributes including vehicle type, occlusion level, and truncation status, enabling evaluation of both detection accuracy and tracking metrics.

7 Evaluation Methodology

We employ a two-tiered evaluation strategy assessing performance at both the tracking level and the analytics level.

7.1 Tracking-Level Metrics

We adopt the standard CLEAR MOT evaluation metrics [19]:

- **Multi-Object Tracking Accuracy (MOTA):**

$$\text{MOTA} = 1 - \frac{\sum_t (\text{FN}_t + \text{FP}_t + \text{IDSW}_t)}{\sum_t \text{GT}_t} \quad (13)$$

- **Identity Switches (IDSW):** Number of times a tracked object’s identity changes.
- **Track Fragmentation (Frag):** Number of times a ground-truth trajectory is interrupted.

7.2 Analytics-Level Metrics

- **Counting Accuracy:** Percentage deviation between estimated and ground-truth vehicle counts over defined time windows and counting lines.
- **Lane Assignment Accuracy:** Fraction of vehicles correctly assigned to their ground-truth lane.
- **Congestion Detection Consistency:** Temporal alignment between system-generated congestion events and manually annotated congestion intervals.

7.3 Success Criteria

We define explicit quantitative targets that constitute success for this project:

Table 2: Quantitative success criteria for system evaluation.

Metric	Target (Good)	Target (Stretch)
MOTA	$\geq 50\%$	$\geq 65\%$
Identity Switches	≤ 200 per sequence	≤ 100 per sequence
Vehicle Counting Accuracy	$\geq 85\%$	$\geq 93\%$
Lane Assignment Accuracy	$\geq 80\%$	$\geq 90\%$
Processing Speed	≥ 15 FPS	≥ 25 FPS (real-time)

The “Good” targets represent the minimum performance level that demonstrates a functional and useful system. The “Stretch” targets represent performance competitive with published results on the UA-DETRAC benchmark. We consider the project successful if all “Good” targets are met.

7.4 Test Conditions

To ensure comprehensive evaluation, we define the following test conditions organized along two axes:

Environmental Conditions: We evaluate on a minimum of 10 UA-DETRAC sequences selected to cover four weather/lighting categories: sunny (3 sequences), cloudy (3 sequences), rainy (2 sequences), and nighttime (2 sequences). Performance is reported both as an aggregate across all conditions and disaggregated per condition to identify environmental weaknesses.

Traffic Density Conditions: Within each weather category, we select sequences spanning low-density (sparse, free-flow traffic), medium-density (moderate traffic with occasional occlusion), and high-density (congested traffic with frequent occlusion) conditions. This yields a minimum of 6 distinct density-environment combinations.

Parameter Sensitivity Conditions: We evaluate the impact of key tunable parameters by sweeping: confidence threshold $\theta_{\text{conf}} \in \{0.3, 0.4, 0.5, 0.6\}$, IoU matching threshold $\theta_{\text{match}} \in \{0.2, 0.3, 0.4\}$, and track termination threshold $n_{\text{lost}} \in \{5, 10, 15, 30\}$ frames.

7.5 Baseline Comparisons

To isolate each component’s contribution, we compare against degraded baselines:

- **No Motion Model:** Replace the Kalman filter with a position-only tracker using the most recent detection as the state estimate.
- **Greedy Association:** Replace the Hungarian algorithm with greedy nearest-neighbor assignment.
- **Detection-Only Counting:** Bypass tracking entirely and count vehicles using per-frame detections with heuristic deduplication.

7.6 Iterative Improvement Strategy

Based on evaluation results, we will apply targeted improvements:

1. **If detection is the bottleneck** (high FN/FP rates): Tune confidence and NMS thresholds; explore fine-tuning YOLOv8 on UA-DETRAC training data.
2. **If identity switches are excessive:** Introduce appearance-based association (DeepSORT-style re-identification features) to supplement IoU-only matching.
3. **If counting accuracy is low despite reasonable tracking:** Re-examine counting line placement; implement trajectory interpolation to fill short detection gaps.
4. **If congestion detection produces false alarms:** Increase temporal smoothing window and tighten speed/density thresholds; evaluate queue length as an alternative indicator.
5. **If processing speed is insufficient:** Profile the pipeline; consider reducing input resolution, increasing detection stride, or switching to a smaller YOLOv8 variant (e.g., YOLOv8n).

8 Expected Challenges and Mitigation Strategies

Occlusion Between Vehicles. In dense traffic, vehicles frequently occlude one another. The Kalman filter’s prediction step mitigates this by propagating state estimates through occlusion periods. The termination threshold n_{lost} must be tuned to balance premature deletion against ghost tracks.

Detection Errors Under Difficult Lighting. Shadows, glare, and nighttime conditions degrade detection performance. We will evaluate sensitivity to the confidence threshold θ_{conf} and explore fine-tuning YOLOv8 on UA-DETRAC training data if needed.

Identity Switches in Dense Traffic. Close-proximity vehicles increase incorrect assignment risk. The Hungarian algorithm reduces this versus greedy matching; appearance features (DeepSORT-style) are a potential extension.

Speed Estimation Inaccuracies. Monocular video without calibration yields pixel-space speeds. These suffice for relative comparisons (congestion detection); absolute speed estimation is a future extension requiring known reference distances or camera calibration.

Scene-Specific Configuration. Counting lines and lane ROIs must be defined per-scene. We mitigate this with a clean configuration interface separating spatial logic from algorithmic logic.

9 Project Timeline

Table 3 presents the planned schedule.

Table 3: Projected timeline for implementation and evaluation.

Weeks	Phase	Activities
1–2	Data & Detection	Download and preprocess UA-DETRAC. Configure YOLOv8 inference. Validate detection quality. Tune confidence and NMS thresholds.
3–5	Tracking	Implement Kalman filter with constant-velocity model. Implement Hungarian algorithm with IoU cost. Develop track life-cycle management.
6–7	Trajectories	Store per-track trajectory histories. Implement smoothing and visualization. Validate trajectory quality.
8–9	Analytics	Implement counting lines, lane ROIs, and congestion detection. Integrate all pipeline components end-to-end.
10	Evaluation	Run full pipeline on UA-DETRAC. Compute all metrics. Execute baseline comparisons and parameter sweeps. Apply iterative improvements.
11	Reporting	Compile results into final report. Prepare presentation. Document code for reproducibility.

10 Conclusion

This proposal presents a modular, vision-based traffic analytics pipeline that transforms raw monocular surveillance footage into structured traffic metrics—vehicle flow rates, lane utilization distributions, and congestion indicators—without dedicated hardware sensors. The system integrates YOLOv8-based vehicle detection, Kalman filter state estimation with Hungarian algorithm data association for tracking, and trajectory-based analytics through configurable spatial primitives.

We plan comprehensive evaluation on the UA-DETRAC benchmark with explicit success criteria, multiple environmental and density test conditions, ablation baselines, and a structured iterative improvement strategy. Future extensions include depth-aware speed estimation, multi-camera fusion, and real-time edge deployment optimization for practical, large-scale smart traffic monitoring.

References

- [1] N. Buch, S. A. Velastin, and J. Orwell, “A review of computer vision techniques for the analysis of urban traffic,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 3, pp. 920–939, 2011.
- [2] S. R. E. Datondji, Y. Dupuis, P. Subirats, and P. Vasseur, “A survey of vision-based traffic monitoring of road intersections,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 10, pp. 2681–2698, 2016.
- [3] C. Stauffer and W. E. L. Grimson, “Adaptive background mixture models for real-time tracking,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 2, pp. 246–252, 1999.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.
- [5] J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” *arXiv preprint arXiv:1804.02767*, 2018.
- [6] G. Jocher *et al.*, “YOLOv5 by Ultralytics.” <https://github.com/ultralytics/yolov5>, 2022.
- [7] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8.” <https://github.com/ultralytics/ultralytics>, 2023.
- [8] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.
- [9] J. Azimjonov, A. Özmen, and M. Varan, “A vision-based real-time traffic flow monitoring system for road intersections,” *Multimedia Tools and Applications*, vol. 82, pp. 9899–9920, 2023.

- [10] W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu, and T.-K. Kim, “Multiple object tracking: A literature review,” *Artificial Intelligence*, vol. 293, p. 103448, 2021.
- [11] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, pp. 3464–3468, 2016.
- [12] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [13] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [14] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, pp. 3645–3649, 2017.
- [15] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-object tracking by associating every detection box,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 1–21, 2022.
- [16] B. Coifman, D. Beymer, P. McLauchlan, and J. Malik, “A real-time computer vision system for vehicle tracking and traffic surveillance,” *Transportation Research Part C: Emerging Technologies*, vol. 6, no. 4, pp. 271–288, 1998.
- [17] B. T. Morris and M. M. Trivedi, “A survey of vision-based trajectory learning and analysis for surveillance,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 18, no. 8, pp. 1114–1127, 2008.
- [18] L. Wen, D. Du, Z. Cai, Z. Lei, M.-C. Chang, H. Qi, J. Lim, M.-H. Yang, and S. Lyu, “UADETRAC: A benchmark suite for multi-object detection and tracking in unconstrained environments,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 982–988, 2020.
- [19] K. Bernardin and R. Stiefelhagen, “Evaluating multiple object tracking performance: The CLEAR MOT metrics,” *EURASIP Journal on Image and Video Processing*, vol. 2008, pp. 1–10, 2008.